



```
In [1]: !pip install nltk scikit-learn
```

```
Requirement already satisfied: nltk in /home/csl-4/anaconda3/lib/python3.9/site-packages (3.7)
Requirement already satisfied: scikit-learn in /home/csl-4/anaconda3/lib/python3.9/site-packages (1.6.1)
Requirement already satisfied: click in /home/csl-4/anaconda3/lib/python3.9/site-packages (from nltk) (8.0.4)
Requirement already satisfied: tqdm in /home/csl-4/anaconda3/lib/python3.9/site-packages (from nltk) (4.64.1)
Requirement already satisfied: joblib in /home/csl-4/anaconda3/lib/python3.9/site-packages (from nltk) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in /home/csl-4/anaconda3/lib/python3.9/site-packages (from nltk) (2022.7.9)
Requirement already satisfied: numpy>=1.19.5 in /home/csl-4/anaconda3/lib/python3.9/site-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /home/csl-4/anaconda3/lib/python3.9/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: scipy>=1.6.0 in /home/csl-4/anaconda3/lib/python3.9/site-packages (from scikit-learn) (1.13.1)
```

```
In [2]: import nltk
import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [3]: nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt to /home/csl-4/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /home/csl-4/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /home/csl-4/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package wordnet to /home/csl-4/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

```
Out[3]: True
```

```
In [4]: doc = """Natural language processing (NLP) is a field of artificial intelligence
that focuses on the interaction between computers and human languages.
The goal of NLP is to enable computers to understand, interpret, and generate
```

```
In [5]: tokens = word_tokenize(doc)
print("Tokens:\n", tokens)
```

Tokens:

```
['Natural', 'language', 'processing', '(', 'NLP', ')', 'is', 'a', 'field', 'o  
f', 'artificial', 'intelligence', '(', 'AI', ')', 'that', 'focuses', 'on', 'th  
e', 'interaction', 'between', 'computers', 'and', 'human', 'languages', '.', 'T  
he', 'goal', 'of', 'NLP', 'is', 'to', 'enable', 'computers', 'to', 'understan  
d', ',', 'interpret', ',', 'and', 'generate', 'human', 'language', '.']
```

```
In [6]: pos_tags = pos_tag(tokens)  
print("\nPOS Tags:\n", pos_tags)
```

POS Tags:

```
[('Natural', 'JJ'), ('language', 'NN'), ('processing', 'NN'), ('(', '(', 'NL  
P', 'NNP'), (')', ')'), ('is', 'VBZ'), ('a', 'DT'), ('field', 'NN'), ('of', 'I  
N'), ('artificial', 'JJ'), ('intelligence', 'NN'), ('(', '(', 'AI', 'NNP'),  
(')', ')'), ('that', 'WDT'), ('focuses', 'VBZ'), ('on', 'IN'), ('the', 'DT'),  
( 'interaction', 'NN'), ('between', 'IN'), ('computers', 'NNS'), ('and', 'CC'),  
( 'human', 'JJ'), ('languages', 'NNS'), ('.', '.'), ('The', 'DT'), ('goal', 'N  
N'), ('of', 'IN'), ('NLP', 'NNP'), ('is', 'VBZ'), ('to', 'TO'), ('enable', 'J  
J'), ('computers', 'NNS'), ('to', 'TO'), ('understand', 'VB'), (',', ','), ('in  
terpret', 'VB'), (',', ','), ('and', 'CC'), ('generate', 'VB'), ('human', 'J  
J'), ('language', 'NN'), ('.', '.')]
```

```
In [7]: stop_words = set(stopwords.words('english'))  
  
filtered_tokens = [word for word in tokens  
                    if word.lower() not in stop_words  
                    and word not in string.punctuation]  
  
print("\nFiltered Tokens:\n", filtered_tokens)
```

Filtered Tokens:

```
['Natural', 'language', 'processing', 'NLP', 'field', 'artificial', 'intellige  
nce', 'AI', 'focuses', 'interaction', 'computers', 'human', 'languages', 'goa  
l', 'NLP', 'enable', 'computers', 'understand', 'interpret', 'generate', 'huma  
n', 'language']
```

```
In [8]: stemmer = PorterStemmer()  
stemmed_tokens = [stemmer.stem(word) for word in filtered_tokens]  
  
print("\nStemmed Tokens:\n", stemmed_tokens)
```

Stemmed Tokens:

```
['natur', 'languag', 'process', 'nlp', 'field', 'artifici', 'intellig', 'ai',  
'focus', 'interact', 'comput', 'human', 'languag', 'goal', 'nlp', 'enabl', 'com  
put', 'understand', 'interpret', 'gener', 'human', 'languag']
```

```
In [9]: lemmatizer = WordNetLemmatizer()  
lemmatized_tokens = [lemmatizer.lemmatize(word) for word in filtered_tokens]  
  
print("\nLemmatized Tokens:\n", lemmatized_tokens)
```

Lemmatized Tokens:

```
['Natural', 'language', 'processing', 'NLP', 'field', 'artificial', 'intellige  
nce', 'AI', 'focus', 'interaction', 'computer', 'human', 'language', 'goal', 'N  
LP', 'enable', 'computer', 'understand', 'interpret', 'generate', 'human', 'lan  
guage']
```

```
In [10]: vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform([doc])

terms = vectorizer.get_feature_names_out()
values = tfidf_matrix.toarray()

print("\nTF-IDF Values:\n")

for i in range(len(terms)):
    print(f"{terms[i]} : {values[0][i]:.4f}")
```

TF-IDF Values:

```
ai : 0.1374
and : 0.2747
artificial : 0.1374
between : 0.1374
computers : 0.2747
enable : 0.1374
field : 0.1374
focuses : 0.1374
generate : 0.1374
goal : 0.1374
human : 0.2747
intelligence : 0.1374
interaction : 0.1374
interpret : 0.1374
is : 0.2747
language : 0.2747
languages : 0.1374
natural : 0.1374
nlp : 0.2747
of : 0.2747
on : 0.1374
processing : 0.1374
that : 0.1374
the : 0.2747
to : 0.2747
understand : 0.1374
```

In []: